

POWER BI



Adventure Work Cycle Report On PowerBI

OBJECTIVE

The easiest start: (Get insights quickly with uncomplicated setup), no training required or included dashboards for services like Salesforce, Google Analytics and Microsoft Dynamics.

Simplified, Publishing and Distribution (SPD).: Emailing large files or placing them on a shared drive, analysts upload reports and representations to Power BI,

This project is completely based on Power BI (business Intelligence) is a set of software services, applications that work together to help you create, share and use business statistics in a way that serves you and your business as effectively as possible. Microsoft Power BI, sometimes referred to as Power BI online, is part of Power BI Software as a Service (SAAS). In the service (Power BI), dashboards help you keep track of every movement of your business. Dashboards display tiles that you can select to open reports for further exploration. (Dashboards and reports connect to datasets that collect all relevant data in one place.

Power BI can provide business intelligence for all industries

1. Power BI brings data to life (interactivity) and easy to understand business.
2. Power BI is secure.
3. Power BI easily connects to many data sources.
4. Power BI has artificial intelligence capabilities.
5. Power BI keeps getting better.
6. Power BI application - excellent means of sharing content.



Introduction to Power BI



Power BI is a stand-alone Business Intelligence product from Microsoft that includes both desktop and web applications for data retrieval and visualization.

More information at powerbi.microsoft.com

AW Cycle Report on Power BI

- Connect and transform, **examine hundreds of thousands** of rows of **records** Access **records** from **clearly everywhere** database tables, flat files, cloud services, Folder etc) and create **absolutely automatic records** shaping and loading (ETL) progresses

- Create relational **fashions** for **blending records** from **more than one** sources Create relationships **among** tables and **examine** holistic **overall performance throughout** the board

Define **complicated** calculations **the usage of records evaluation** Experiences (DAX)

Enhance datasets and **permit superior evaluation** with **effective** and DAX **transportable** expressions

- Visualize **records the usage of** interactive **reviews** and dashboards Build **custom commercial enterprise** intelligence **gear** with best-in-magnificence visualization And dashboard features

- Power BI is the **enterprise chief amongst** BI platforms Microsoft Power BI is intuitive, **effective** and **absolutely** FREE Started.

Problem statement

1. Their 1 Data are refreshed each time the base dataset is updated.
2. Real-time information: The dashboard updates in real-time as data is sent or streamed, giving viewers the ability to quickly solve problems and identify opportunities. If any report or dashboard can be viewed and updated, it can be real-time data and visuals (sensors), social media feeds, or anything from which time-sensitive data can be collected.
3. Customization option (MS Power BI application navigation) The "in-app navigation" feature gives report developers the ability to customize navigation to help viewers quickly find content and understand the relationships between different reports and dashboards.
4. Customize security features: Report developers can set row-level security filters to ensure viewers only see data that is relevant to them, reducing the risk of people seeing data they shouldn't.
5. Cortana Integration (CI): Using Power BI with Microsoft's digital assistant Cortana. A specific user can easily verbally ask a question in natural language and access tables and graphs. This will be especially useful for users with mobile devices.
6. Artificial Intelligence: (AI):MS Power BI users can access (image recognition) and text analytics, build machine learning models and integrate with Azure Machine Learning.

About the dataset

MS Power BI Premium is suitable for enterprise-wide businesses with advanced analytics requirements such as working with (big data), creating calculated entities in the data stream, leveraging AI features, and requiring high performance for big data analytics.

Login: shows the users Power BI login.

- **Bio:** about the user's interest.
- **Blog:** show the link to the users block side.
- **Company:** user connected with the organization.
- **Email:** email id of the user.
- **Id:** users id.
- **Location:** locations where the user sides.
- **Name:** Full Name of the users.
- **url:** API link of the user's profile.

SYSTEM REQUIREMENT SPECIFICATION

HARDWARE SPECIFICATION

VIRTUAL MEMEORY

PROCESSOR : 64 BIT, Pentium – IV

RAM :GB

HARD DISK : 1 GB(minimum)

MONITOR : 15inch (1920 *1080 RESOLUTIONS)

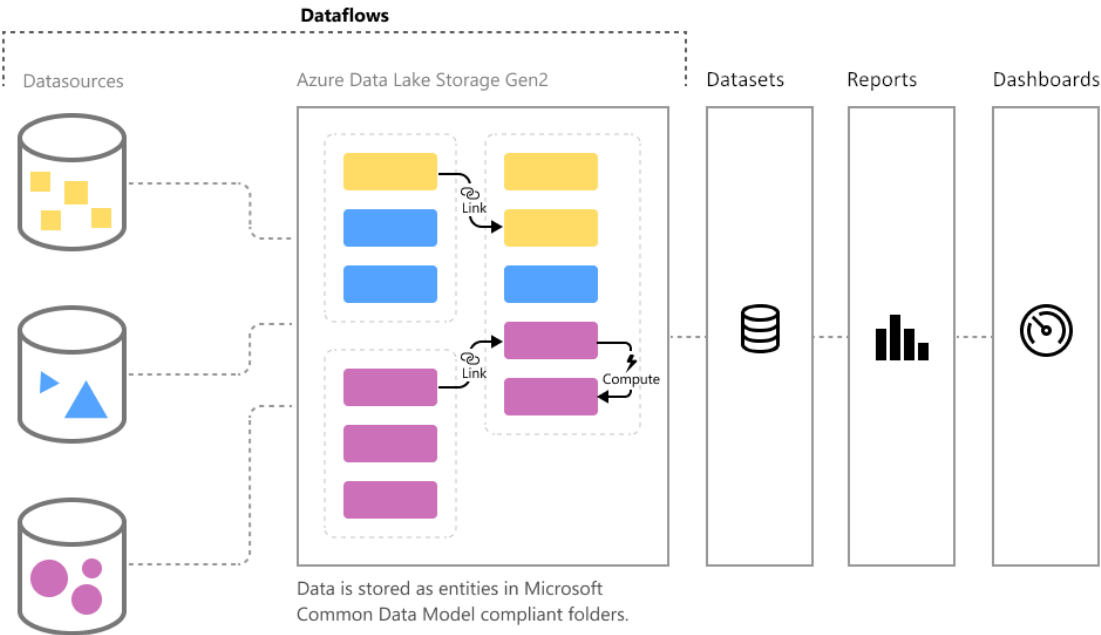
CLOCK SPEED : 1.6 GHz - 3.4 GHz

SOFTWARE SPECIFICATION

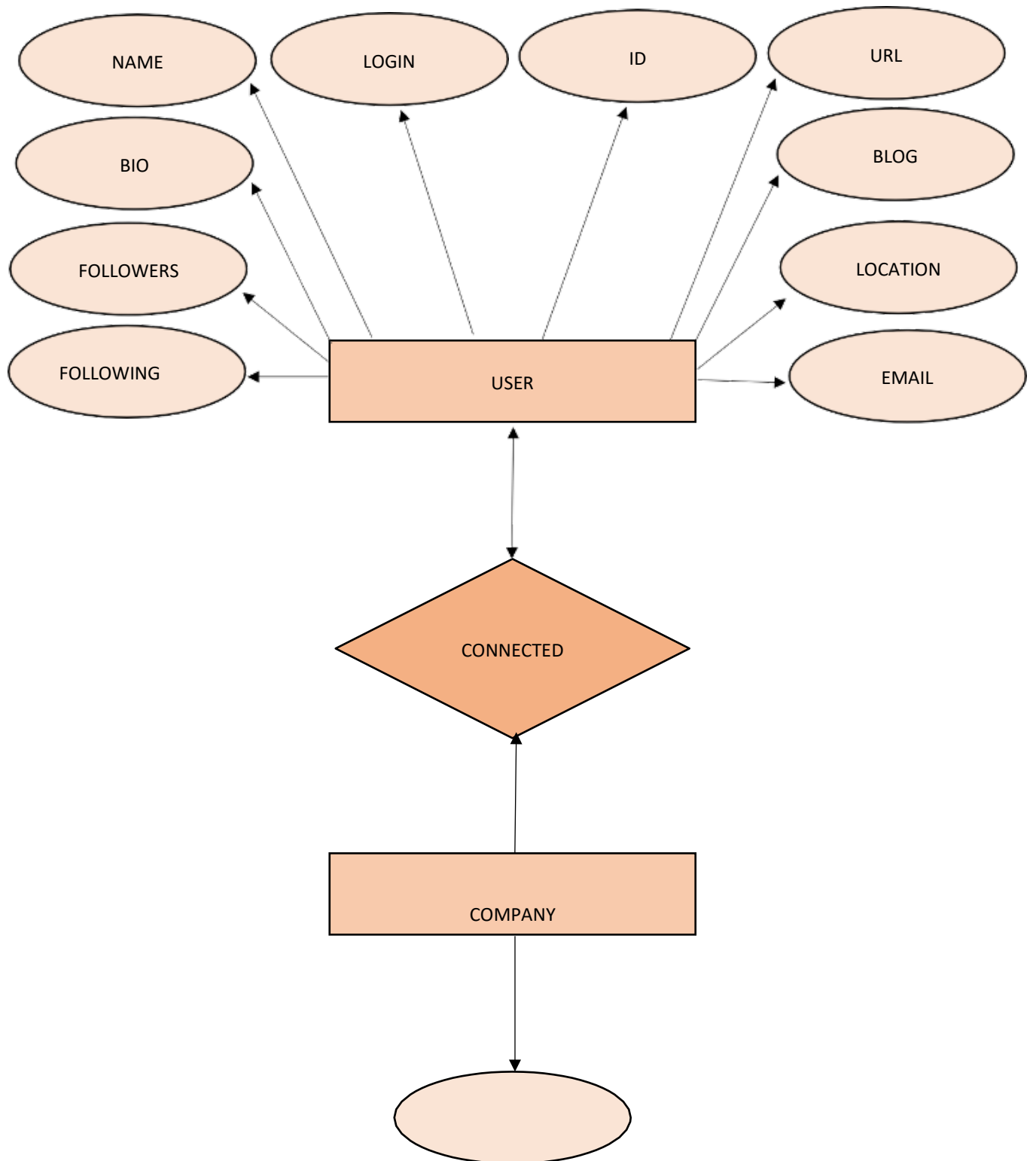
OPERATING SYSTEM : 64 Bit operating system, x64-based
processor

APPLICATION PROGRAM : Power BI

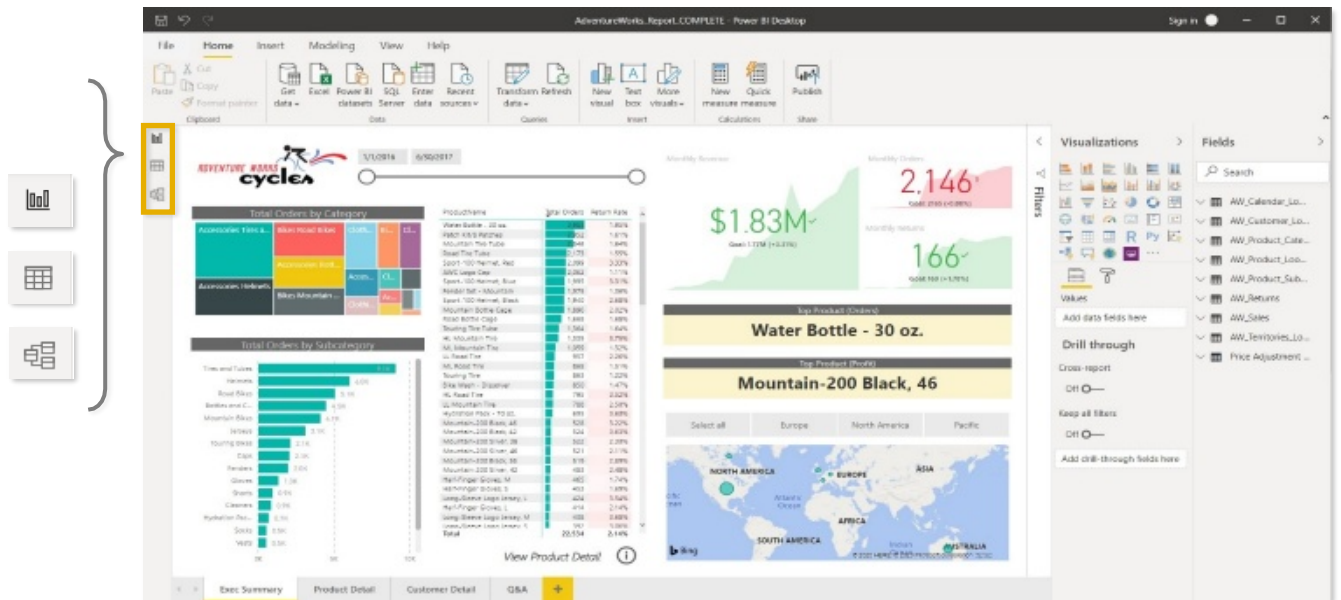
Data flow diagram of Power BI



E-R DIAGRAM

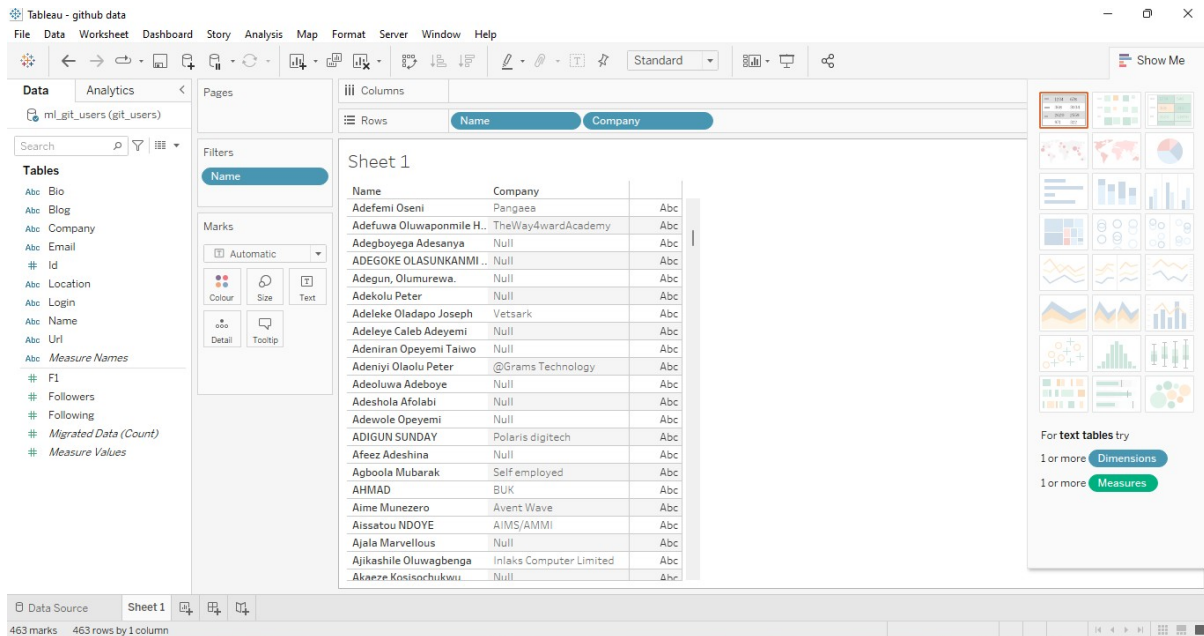


Dashboard

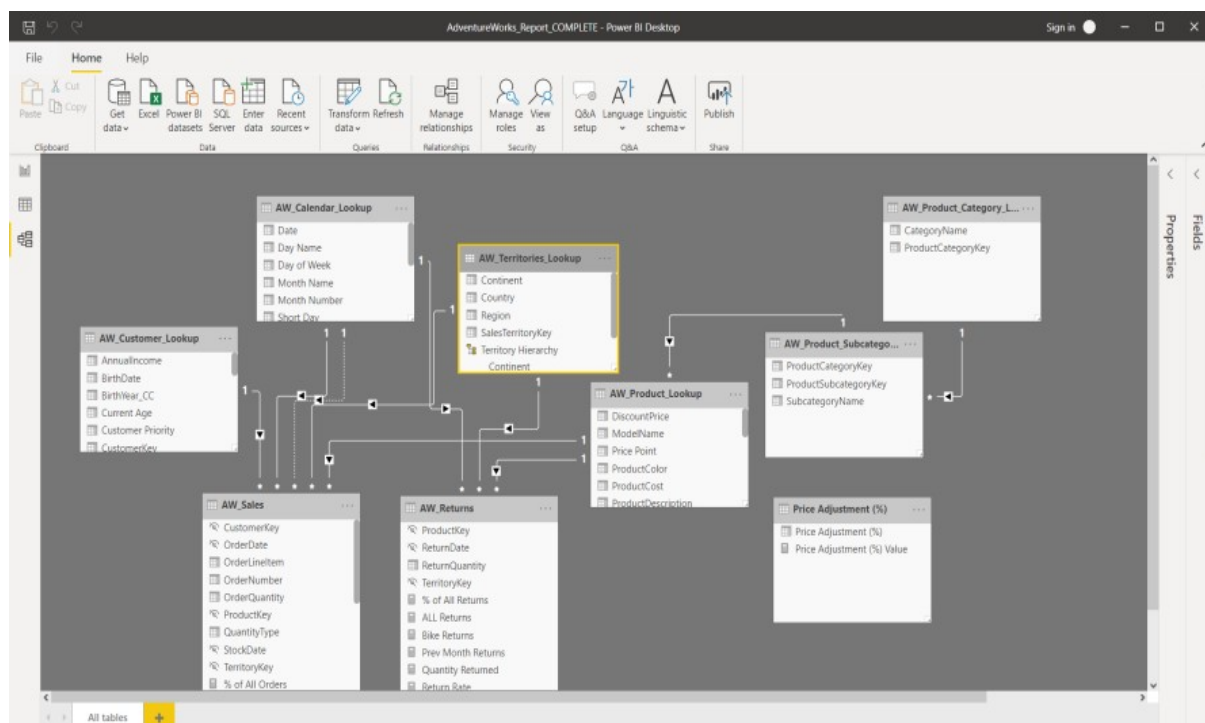


Data model

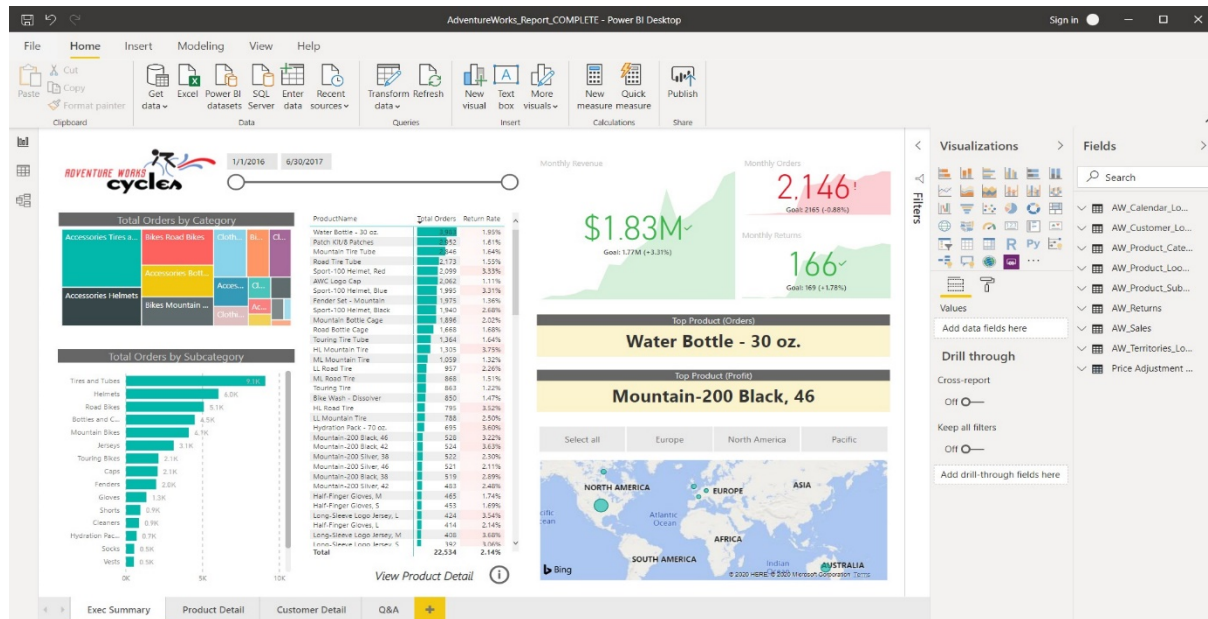
Connect, form and remodel transform information



Build **table** relationships to create a **information** model

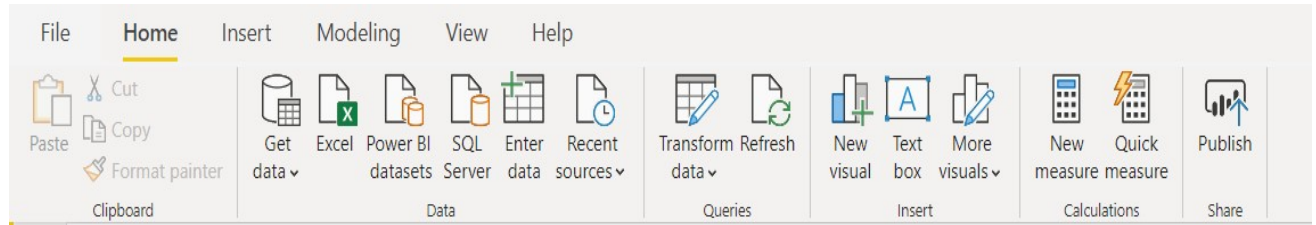


The interactive Design reviews to **discover** and Visualize Data



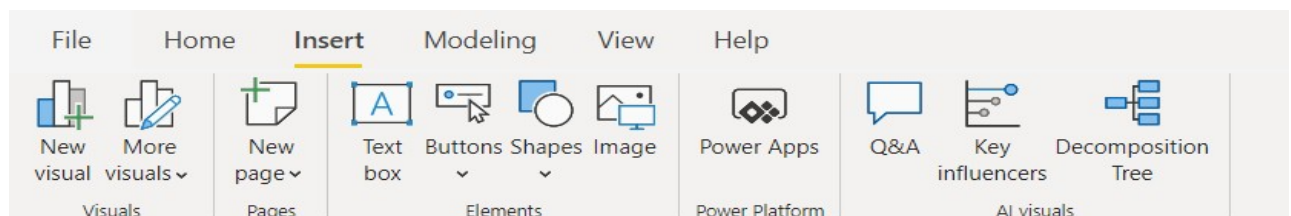
Report View Tabs

HOME



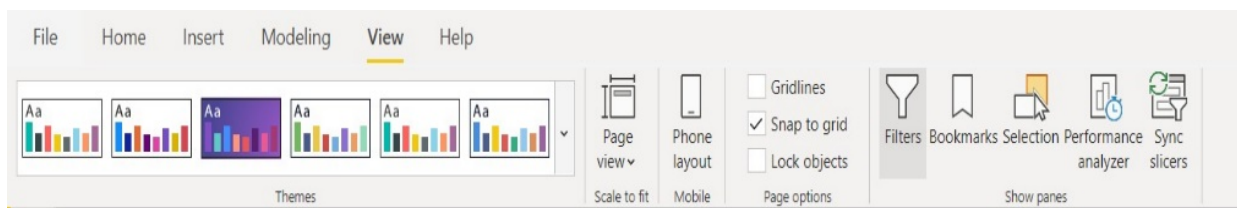
HOME: The authentic Home equipment are actually cut up throughout Home & Insert .

INSERT



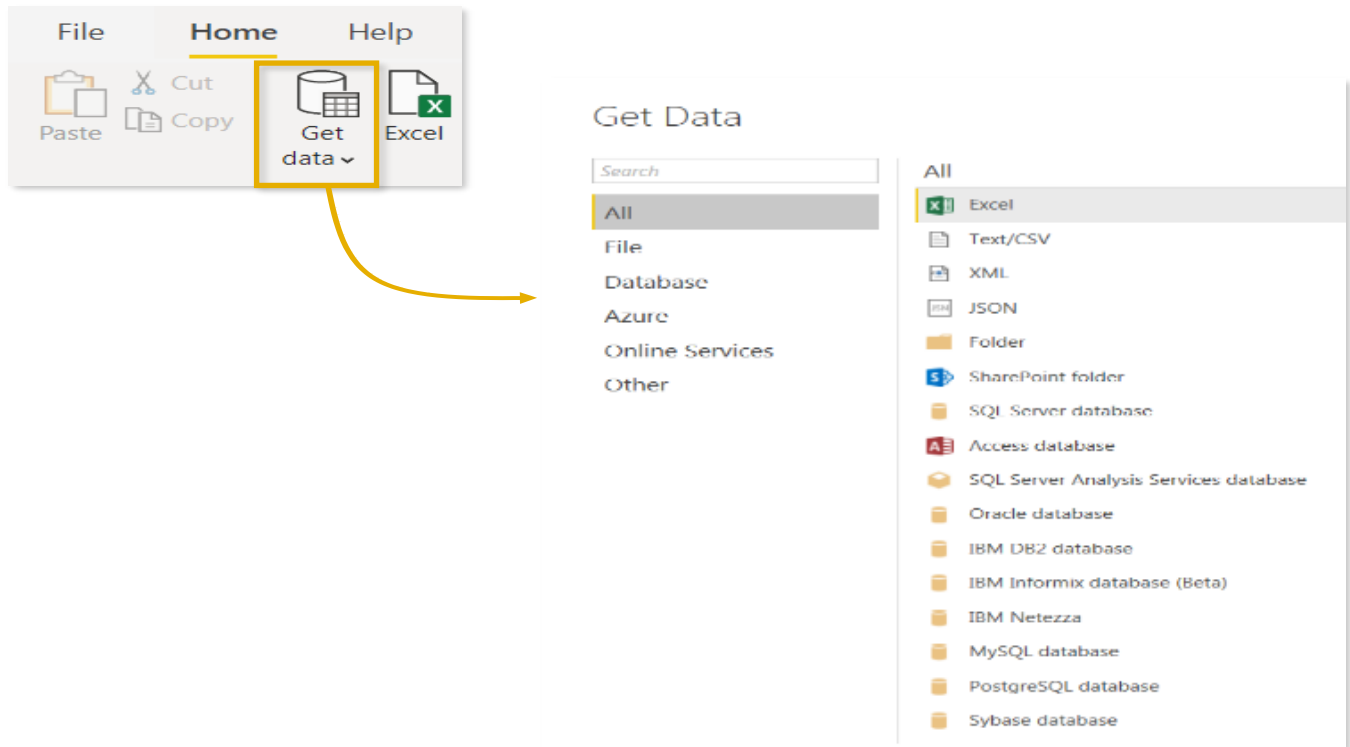
Power Apps and AI Visuals are actually featured withinside the Insert menu.

VIEW



Performance analyzer totune & optimize reports

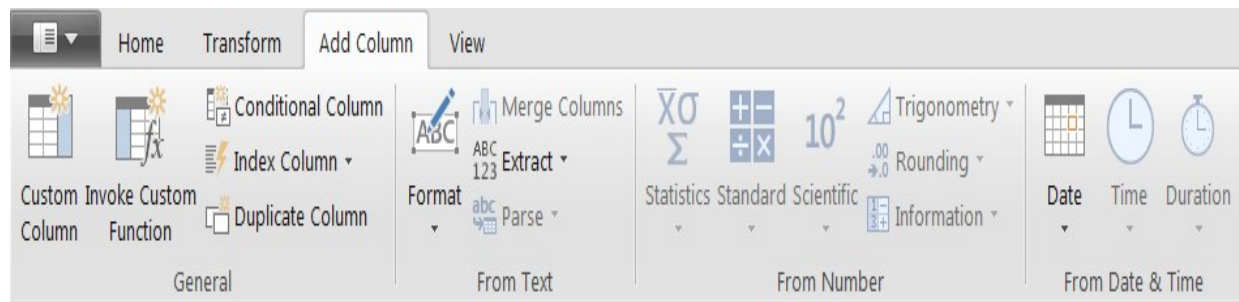
Connecting and Shaping Data



Types of Data Connectors

MS Power BI can connect virtually any type of source data, including (but not limited to):

- Flat files and folders (CSV, TEXT, XLS, etc.)
- DATABASE (SQL, ORACLE, IBM, AZURE, etc.)
- ONLINE SERVICES (SHARE BOD, GITHUB, POWER BI SERVICEDYNAMIC 365, GOOGLE ANALYSYS)
- OTHERS (WEB RESOURCES, R SCRIPTS, SPARK, HADOOP, etc.)



- The day and time **equipment** are **fantastically trustworthy** and **encompass the subsequent alternatives**:
- Age: The difference between the current time and the day in each row
- Age: The **distinction among** the **modern-day** time and the day in **every** row
- Date Only: Removes the time **factor** from the date/time field
 Year/Month/Quarter/Week/Day: Extracts **person additives** from a date field (time-**unique alternatives encompass** hour, minute, second, etc.)
 or Newest: Evaluates the (oldest or newest) date from a column as a value
 (can **handy most effective** from the "Transform" menu)

Note: You will **nearly constantly need** to **carry out those** operations from the "Add Columns" menu to

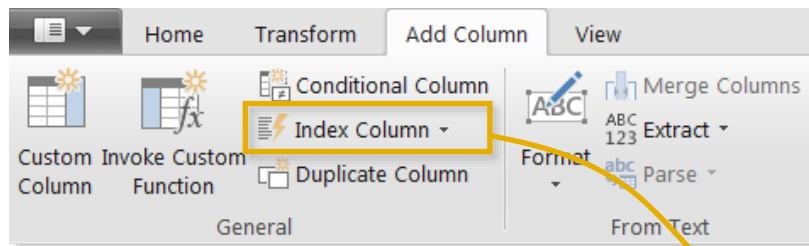
create new fields **rather than reworking person** datetime columns

Creating A Basic Calender Table

	Date	1 ² ₃ Day	1 ² ₃ Day of Week	A ^B _C Day Name	Start of Week	1 ² ₃ Month
1	1/1/2015	1	4	Thursday	12/28/2014	1
2	1/2/2015	2	5	Friday	12/28/2014	1
3	1/3/2015	3	6	Saturday	12/28/2014	1
4	1/4/2015	4	0	Sunday	1/4/2015	1
5	1/5/2015	5	1	Monday	1/4/2015	1
6	1/6/2015	6	2	Tuesday	1/4/2015	1
7	1/7/2015	7	3	Wednesday	1/4/2015	1
8	1/8/2015	8	4	Thursday	1/4/2015	1
9	1/9/2015	9	5	Friday	1/4/2015	1
10	1/10/2015	10	6	Saturday	1/4/2015	1
11	1/11/2015	11	0	Sunday	1/11/2015	1
12	1/12/2015	12	1	Monday	1/11/2015	1
13	1/13/2015	13	2	Tuesday	1/11/2015	1
14	1/14/2015	14	3	Wednesday	1/11/2015	1
15	1/15/2015	15	4	Thursday	1/11/2015	1
16	1/16/2015	16	5	Friday	1/11/2015	1
17	1/17/2015	17	6	Saturday	1/11/2015	1
18	1/18/2015	18	0	Sunday	1/18/2015	1
19	1/19/2015	19	1	Monday	1/18/2015	1
20	1/20/2015	20	2	Tuesday	1/18/2015	1
21	1/21/2015	21	3	Wednesday	1/18/2015	1
22	1/22/2015	22	4	Thursday	1/18/2015	1
23	1/23/2015	23	5	Friday	1/18/2015	1
24	1/24/2015	24	6	Saturday	1/18/2015	1
25	1/25/2015	25	0	Sunday	1/25/2015	1

- Predefined **Dates**, option in the “**Add Columns**” menu to quickly build, out a calendar table from a list of dates.

ADDING INDEX COLUMN

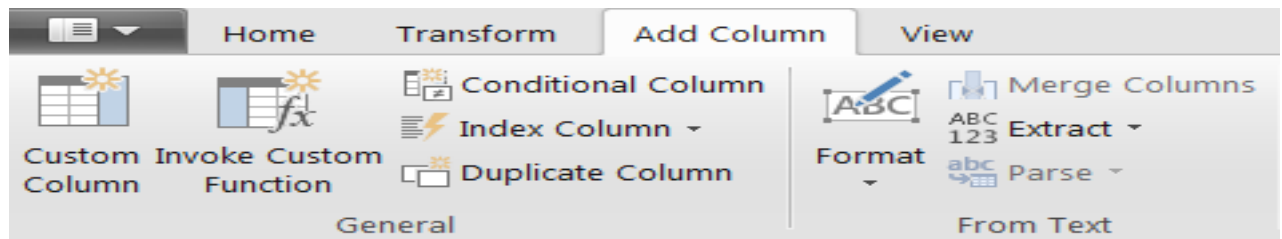


	1 ² ₃ Index	OrderDate	StockDate	A ^B _C OrderNumber	1 ² ₃ ProductKey	1 ² ₃ CustomerKey
1	1	1/1/2015	9/21/2001	SO45080	332	14657
2	2	1/1/2015	12/5/2001	SO45079	312	29255
3	3	1/1/2015	10/29/2001	SO45082	350	11455
4	4	1/1/2015	11/16/2001	SO45081	338	26782
5	5	1/2/2015	12/15/2001	SO45083	312	14947
6	6	1/2/2015	10/12/2001	SO45084	310	29143
7	7	1/2/2015	12/18/2001	SO45086	314	18747
8	8	1/2/2015	10/9/2001	SO45085	312	18746
9	9	1/3/2015	10/3/2001	SO45093	312	18906
10	10	1/3/2015	9/29/2001	SO45090	310	29170
11	11	1/3/2015	12/11/2001	SO45088	345	11398
12	12	1/3/2015	10/24/2001	SO45092	313	18899
13	13	1/3/2015	12/16/2001	SO45089	351	25977
14	14	1/3/2015	10/26/2001	SO45091	314	18909
15	15	1/3/2015	9/11/2001	SO45087	350	11388
16	16	1/3/2015	9/11/2001	SO45094	310	22785
17	17	1/4/2015	10/30/2001	SO45096	312	12483
18	18	1/4/2015	10/30/2001	SO45097	313	29151

- Index columns contain a list of sequential values that can be used to identify each unique row in the table (usually starting from 0 or 1).
- These columns are often used to create unique IDs that can be used to create relationships between tables more on that.

ADDING CONDITIONAL COLUMN

A **Conditional _Column** allows you to define new fields based on logical rules and conditions (*IF/THEN statements*).



In this case, we create a new conditional column called "QuantityType" that depends on the values in the "OrderQuantity" column as follows:

- If *OrderQuantity* = 1, *QuantityType* = "One Item"
- If *OrderQuantity* > 1, *QuantityType* = "Multiple items"
- Else *QuantityType* = "Other"

Add Conditional Column

Add a conditional column that is computed from the other columns or values.

New column name

QuantityType

	Column Name	Operator	Value		Output
If	OrderQuantity	equals	1	Then	Single Item
Else If	OrderQuantity	is greater than	1	Then	Multiple Items

Add rule

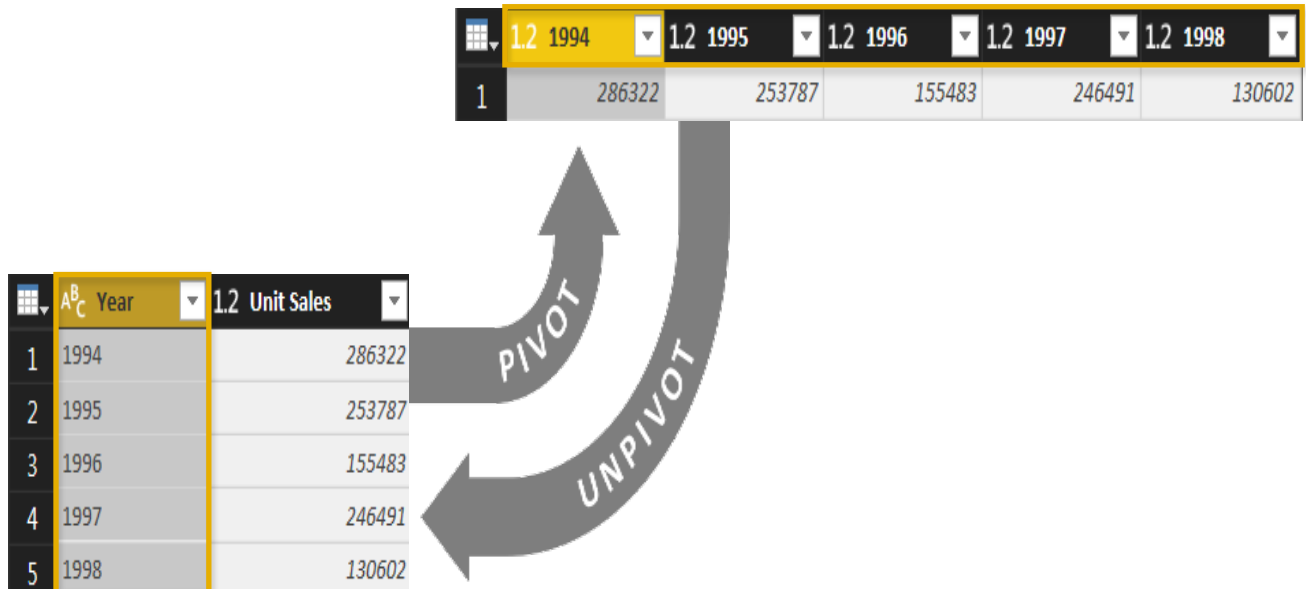
Otherwise

Other

OK

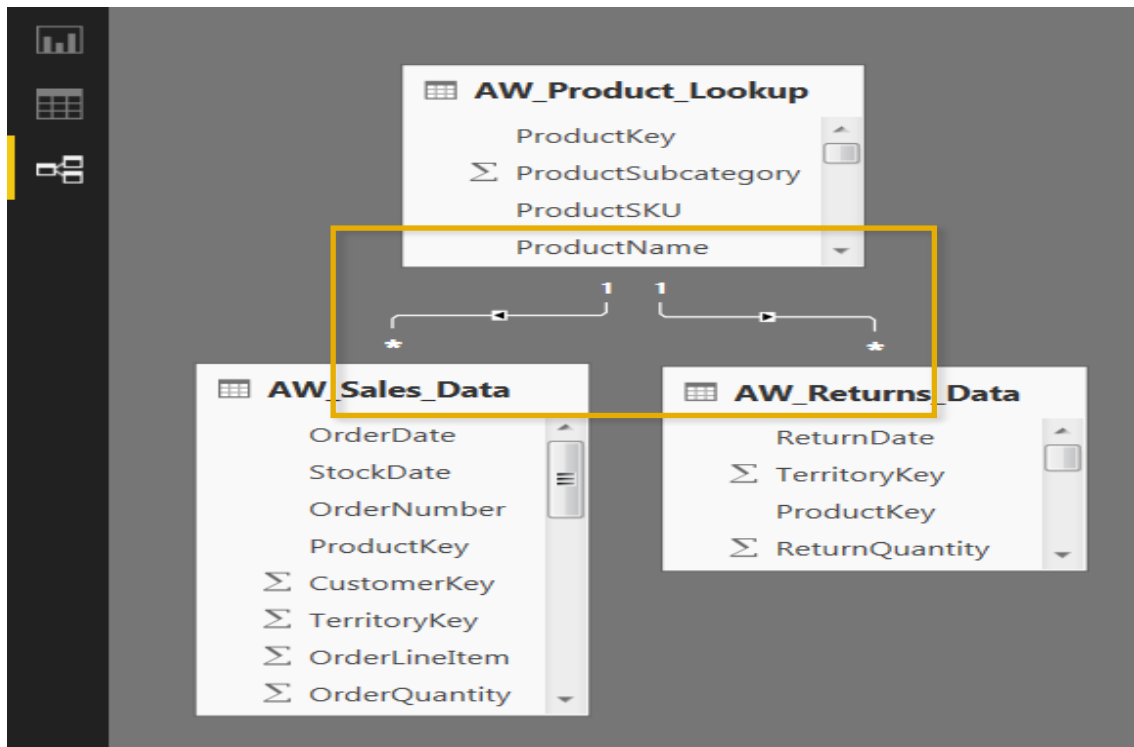
Cancel

PIVOTING & UNPIVOTING



- Imagine that the **desk** is on a hinge; pivoting is like rotating it from a vertical to a horizontal layout, and unpivoting is like rotating it from horizontal to vertical
- Imagine that the **desk** is on a hinge; pivoting is like rotating it from a vertical to a horizontal layout, and unpivoting is like rotating it from horizontal to Vertical.

Data Model



- This is a data model.
- Tables are linked through a relationship based on the Common Product Key field
- Now sales and returns tables can be filtered using fields from the products table

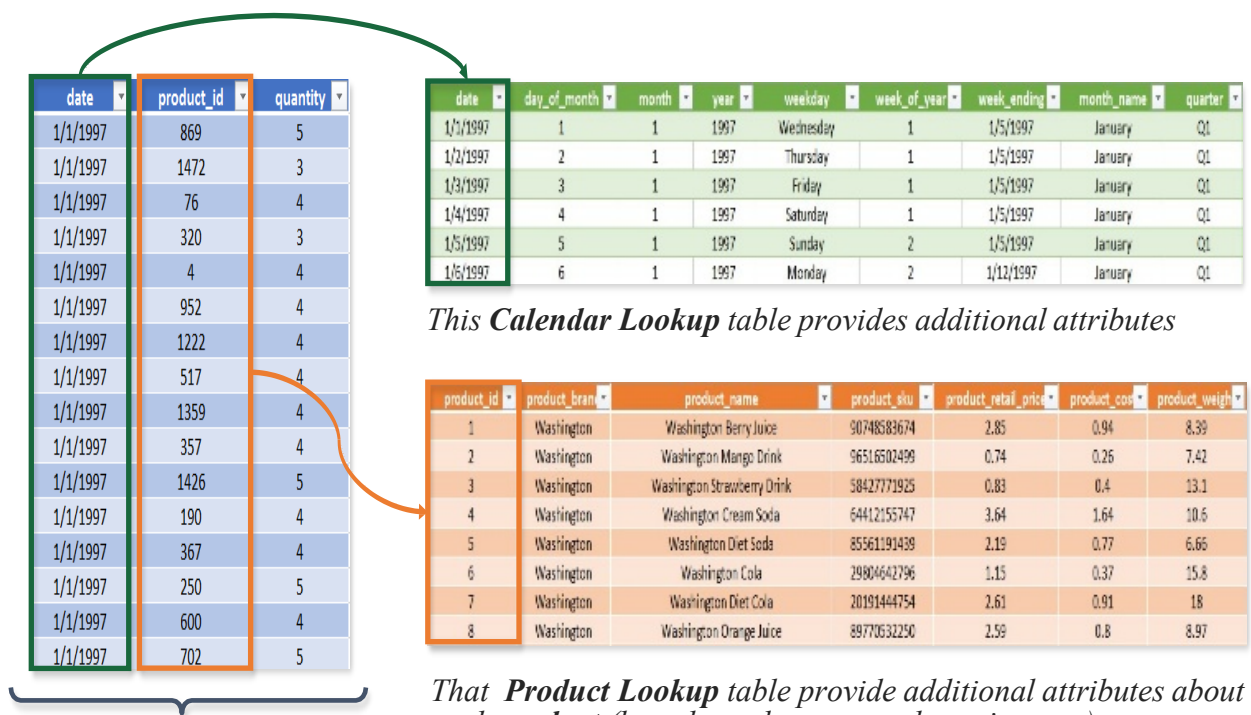
Database Normalization

- A Normalization is the manner of organizing the tables and columns in a relational database to lessen redundancy and maintain facts integrity. It's typically used to:
 - Eliminate redundant facts to lower desk sizes and enhance processing speed & efficiency
 - Minimizing mistakes and anomalies from facts modifications (inserting, updating or deleting records)
 - Simplify queries and shape the database for significant analysis

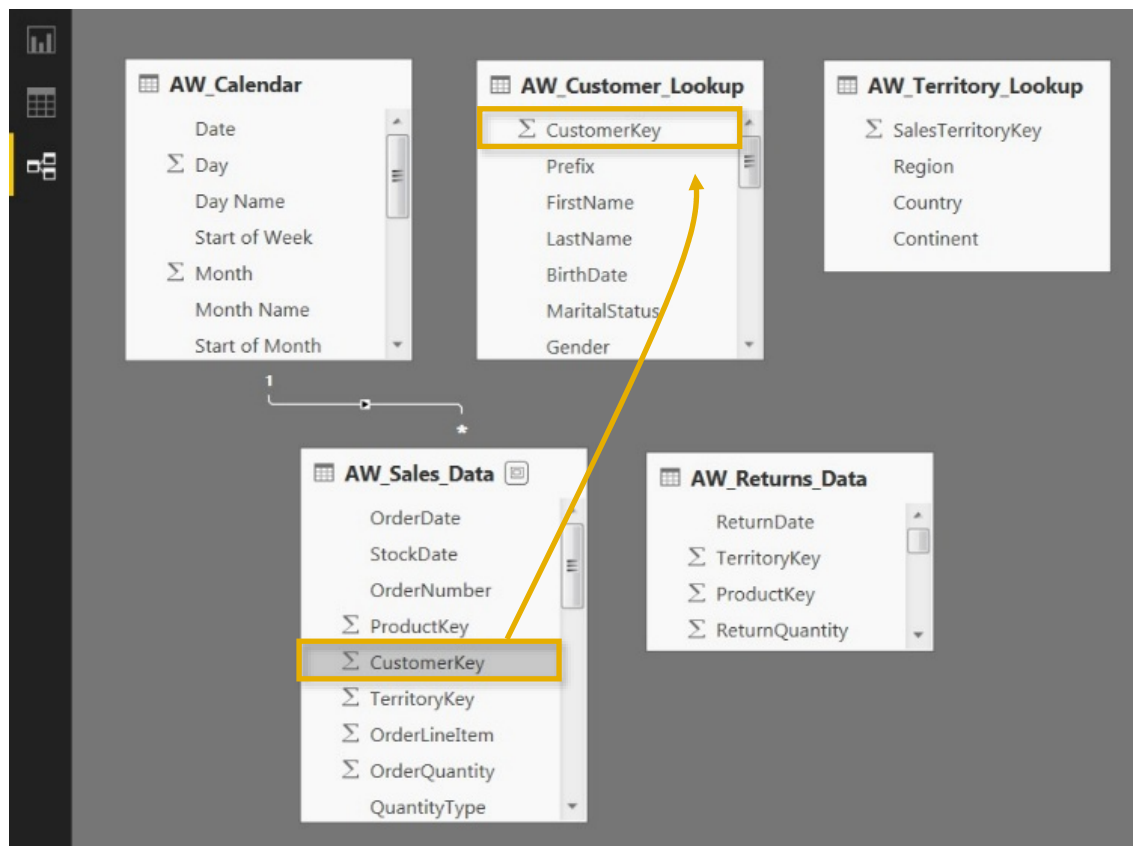
date ▾	product_id ▾	quantity ▾	product_brand ▾	product_name ▾	product_sku ▾	product_weight ▾
1/1/1997	869	5	Nationeel	Nationeel Grape Fruit Roll	52382137179	17
1/7/1997	869	2	Nationeel	Nationeel Grape Fruit Roll	52382137179	17
1/3/1997	1	4	Washington	Washington Berry Juice	90748583674	8.39
1/1/1997	1472	3	Fort West	Fort West Fudge Cookies	37276054024	8.28
1/6/1997	1472	2	Fort West	Fort West Fudge Cookies	37276054024	8.28
1/5/1997	2	4	Washington	Washington Mango Drink	96516502499	7.42
1/1/1997	76	4	Red Spade	Red Spade Sliced Chicken	62054644227	18.1
1/1/1997	76	2	Red Spade	Red Spade Sliced Chicken	62054644227	18.1
1/5/1997	3	2	Washington	Washington Strawberry Drink	58427771925	13.1
1/7/1997	3	2	Washington	Washington Strawberry Drink	58427771925	13.1
1/1/1997	320	3	Excellent	Excellent Cranberry Juice	36570182442	16.4

Lookup Vs. Data Tables

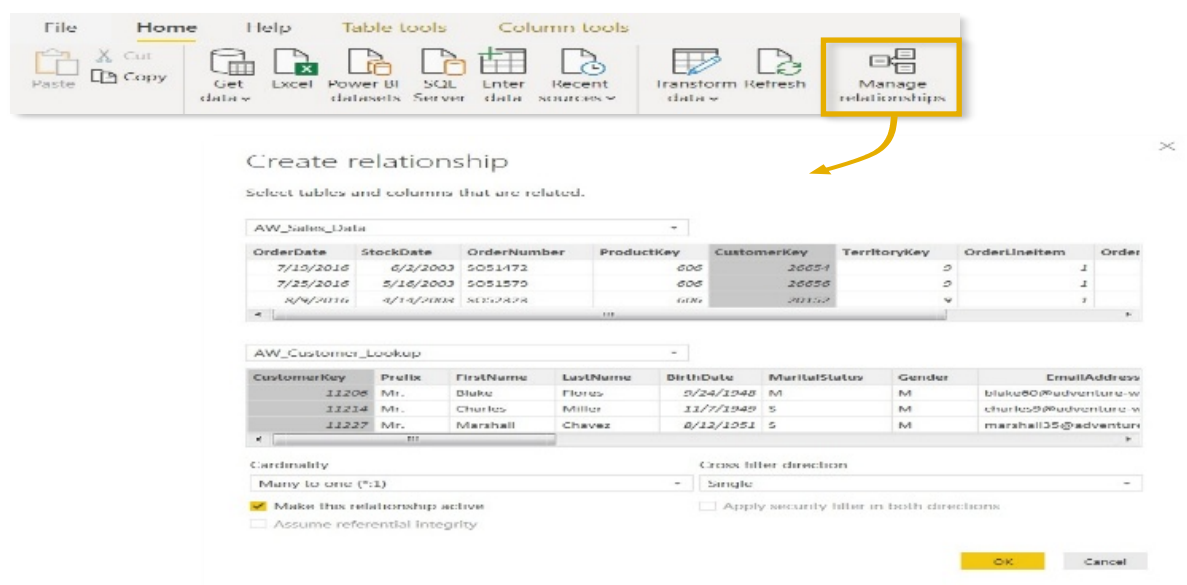
- A Model generally contain two types of tables **data** (or “*fact*”) tables, and **lookup** (or “*dimension*”) tables
 - **Data table contains** *numbers* or *values*, typically at a granular level, with ID or “*key*” columns that can be use to create table relationships
 - **Lookup tables** provide descriptive, often text-based *attributes* about each dimension in the table.



Creating Table Relationship

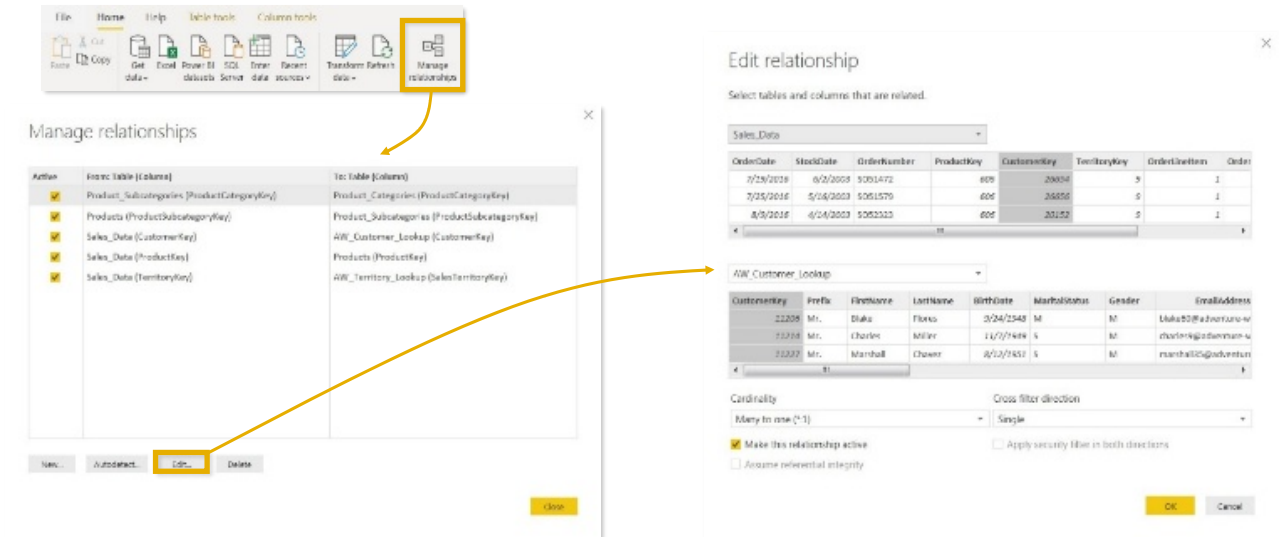


Option 1: Click and drag to connect primary and foreignkeys within the Relationship pane

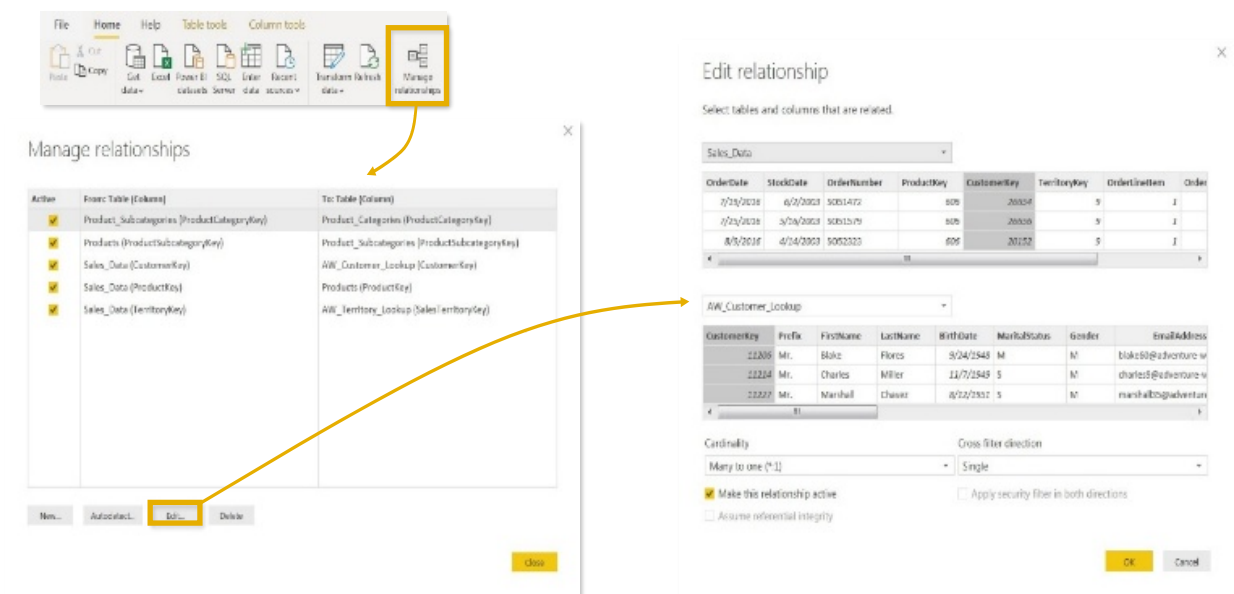


Option 2: Add or detect relationships using the “Manage Relationships” dialog box.

Managing & Editing Relationship



The “**Manage Relationships**” dialog box allows you to **add, edit, or delete** table relationships



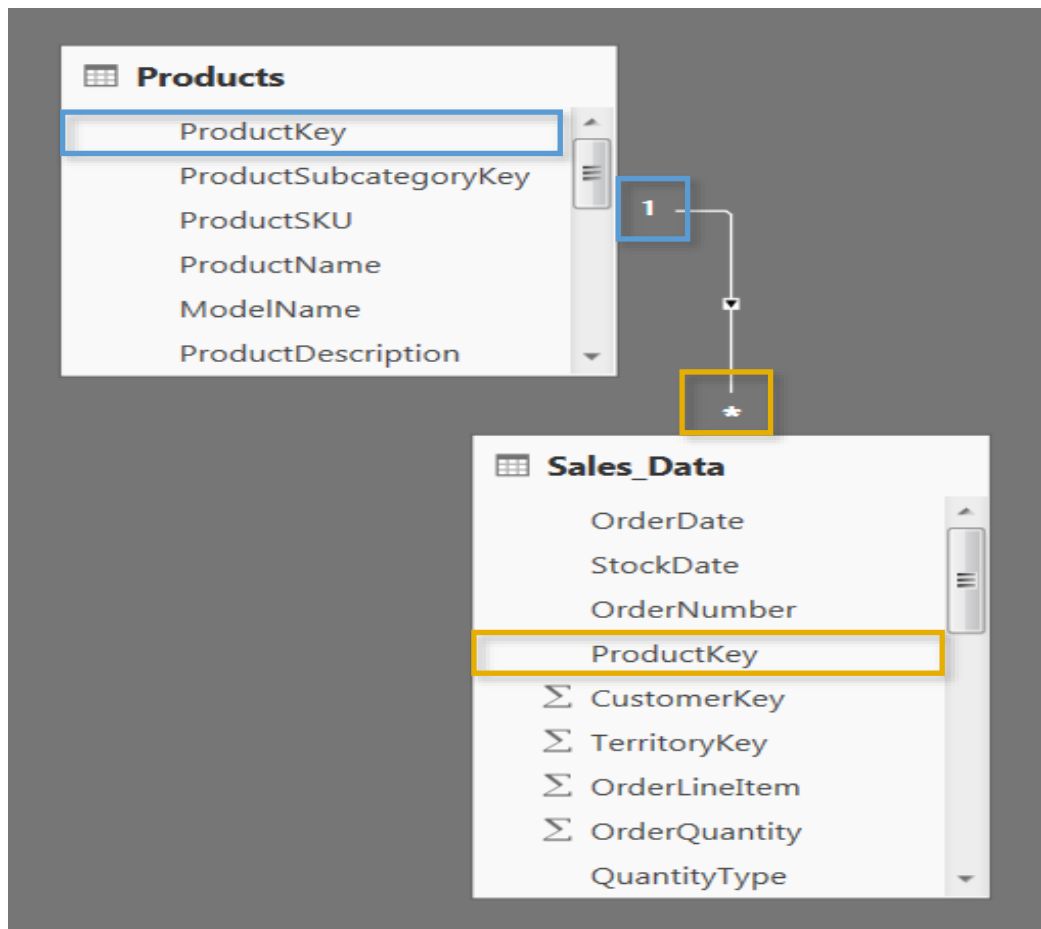
This Editing tool allow you to **activate/deactivate** relationships, view **cardinality**, and modify the **cross filter direction** (stay tuned!)

Active Vs Inactive Relationship

The screenshot illustrates the relationship between the **Calendar** and **Sales_Data** tables in Power BI. The **Calendar** table contains date-related fields, and the **Sales_Data** table contains sales transaction data. The relationship between **OrderDate** in **Sales_Data** and **Date** in **Calendar** is highlighted. The **Edit relationship** dialog is open, showing the **Make this relationship active** checkbox checked, indicating that this is the active relationship.

- These **Sales**, tables contains two date fields, (1-**OrderDate** & 2-**StockDate**), but there can only be one (*active* _relationships) to the Date fields, in the Calendar table
- First Double-click those relationship lines, and check the “**Makes this relationships active**” box to toggle (note that you have to deactivate one in order to activate to others)

Relationship Cardinality



Cardinality refer to a unique of values in a columns **withinside the** Table

or our purposes, all relationships **withinside the information version ought to comply with** a “one-to-many” cardinality; one **example of every number one** key, **however doubtlessly many times of every foreign key**

In this case, **there's only ONE times of every ProductKeys withinside the every** Products table (**mentioned via way of means of the “1”**), **considering that every row includes** attributes of a **unmarried** product (Name, SKU, Description, Retail Price, etc)

There are so **MANY times of every ProductKey withinside the Sales Data table** (**mentioned via way of means of the asterisk ***), **considering that there are a couple of income related to every product**

Cardinality Case Study: Many To Many

product_id	product_name	product_sku
4	Washington Cream Soda	64412155747
4	Washington Diet Cream Soda	81727382373
5	Washington Diet Soda	85561191439
7	Washington Diet Cola	20191444754
8	Washington Orange Juice	89770532250

date	product_id	transactions
1/1/2017	4	12
1/2/2017	4	9
1/3/2017	4	11
1/1/2017	5	16
1/2/2017	5	19
1/1/2017	7	11

Create relationship

You can't create a relationship between these two columns because one of the columns must have unique values.

OK

- If We try to connect these tables using **product id**, we'll get a “*many-to-many relationship*” error since there are multiple instances of each ID in both tables.
-
- *Even if we* create this relationship, how wouldyou know which product was actually sold on each date – ***Cream Soda*** or ***Diet Cream Soda***?

Cardinality Case Study :One To One

product_id	product_name	product_sku
4	Washington Cream Soda	64412155747
5	Washington Diet Soda	85561191439
7	Washington Diet Cola	20191444754
8	Washington Orange Juice	89770532250

product_id	product_price
4	\$3.64
5	\$2.19
7	\$2.61
8	\$2.59

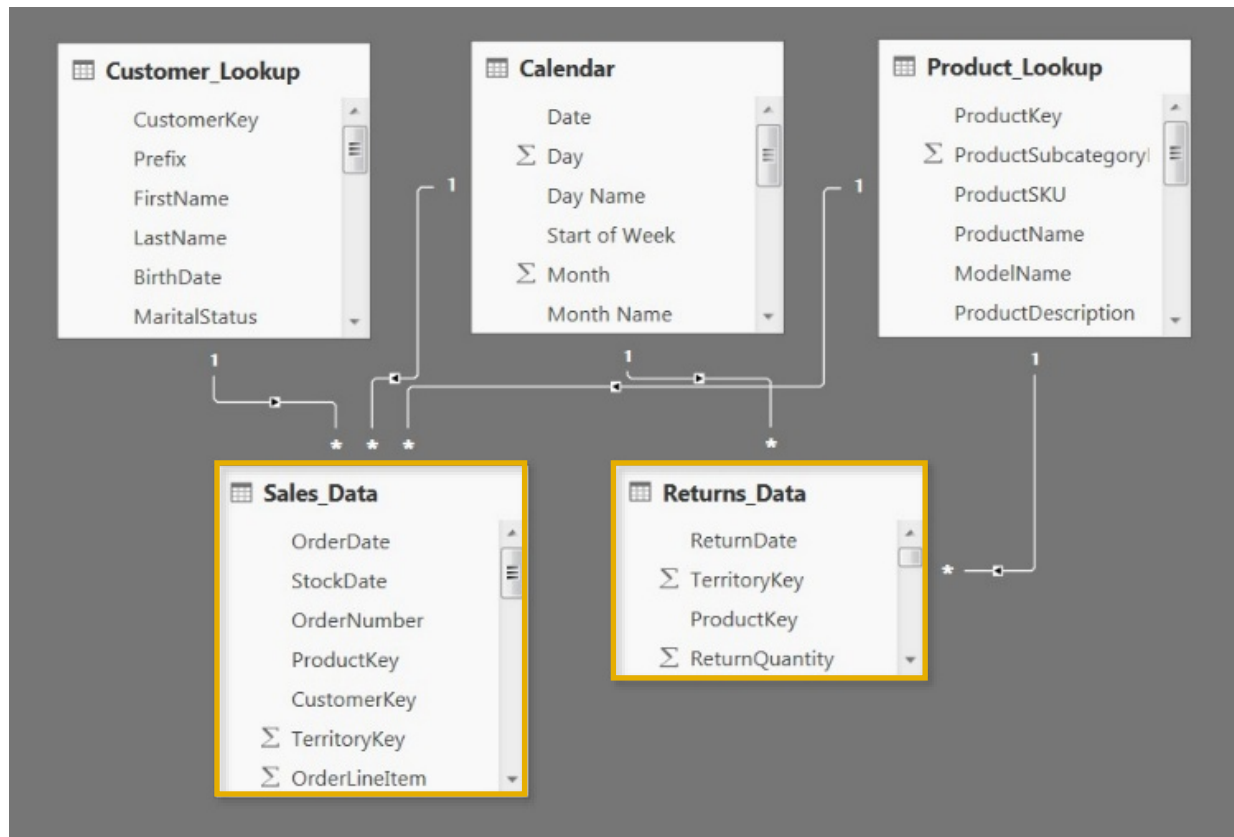
- Joining the above tables the use of the Product ID subject creates a one-to-one coupling due to the fact every ID takes place most effectively as soon as in every table.
- Unlike many-to-many, this coupling isn't illegal.

product_id	product_name	product_sku	product_price
4	Washington Cream Soda	64412155747	\$3.64
5	Washington Diet Soda	85561191439	\$2.19
7	Washington Diet Cola	20191444754	\$2.61
8	Washington Orange Juice	89770532250	\$2.59

- To do away with inefficiencies, you could in reality be part of tables into one legitimate lookup.

product_id	product_name	product_sku	product_price
4	Washington Cream Soda	64412155747	\$3.64
5	Washington Diet Soda	85561191439	\$2.19
7	Washington Diet Cola	20191444754	\$2.61
8	Washington Orange Juice	89770532250	\$2.59

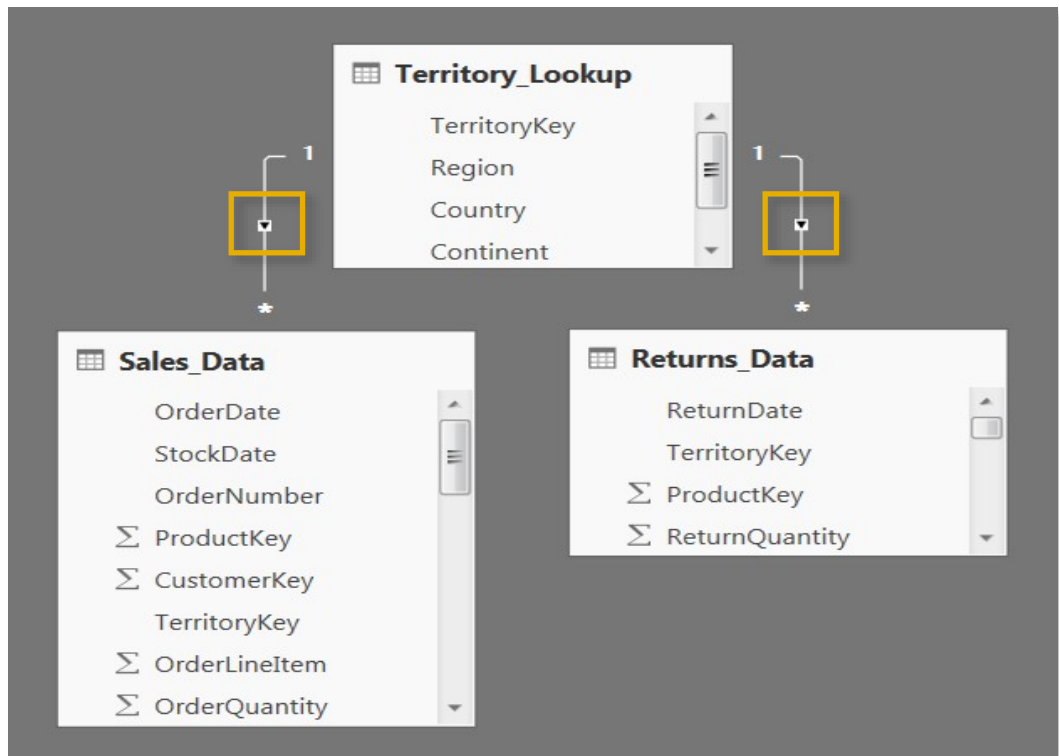
Connecting Multiple Table



This model Contains two data tables: Sales Data and Return Data.

- Note that the Returns table Connects to Calendar and product lookup just like the Sales table, but without a Customer key field it can not be joined to Customer Lookup.
- This is allows us to analyze sales and returns within the same view, but only if we filter or segment the data using shared lookups.
- In other words, we know products was return with in the same view, but only if we filter or Segment the data using shared lookups.
- In other words, we know which products was returned and on which dated, but nothing about which customer made the return.

FilterFlow



Two-Way Filter

The image shows the 'Edit relationship' dialog in a data tool, with a yellow arrow pointing from the 'Both' option in the 'Cross filter direction' dropdown to a 'Both' filter icon in a data model diagram.

Edit relationship dialog:

Select tables and columns that are related.

Sales Data

OrderDate	StockDate	OrderNumber	ProductKey	CustomerKey	TerritoryKey	OrderLineItem	Order
7/29/2016	8/22/2009	5051472		605	26550	9	1
7/29/2016	8/22/2009	5051571		605	26550	9	1
8/9/2016	4/14/2009	5052523		605	26152	9	1

Territory_Lookup

TerritoryKey	Region	Country	Continent
1	Northwest	United States	North America
2	Northeast	United States	North America
3	Central	United States	North America

Cardinality: Many to one (*:1)

☒ Make this relationship active

☐ Assume referential integrity

Cross filter direction: **Both**

☐ Apply security filter in both directions

OK Cancel

Data Model Diagram:

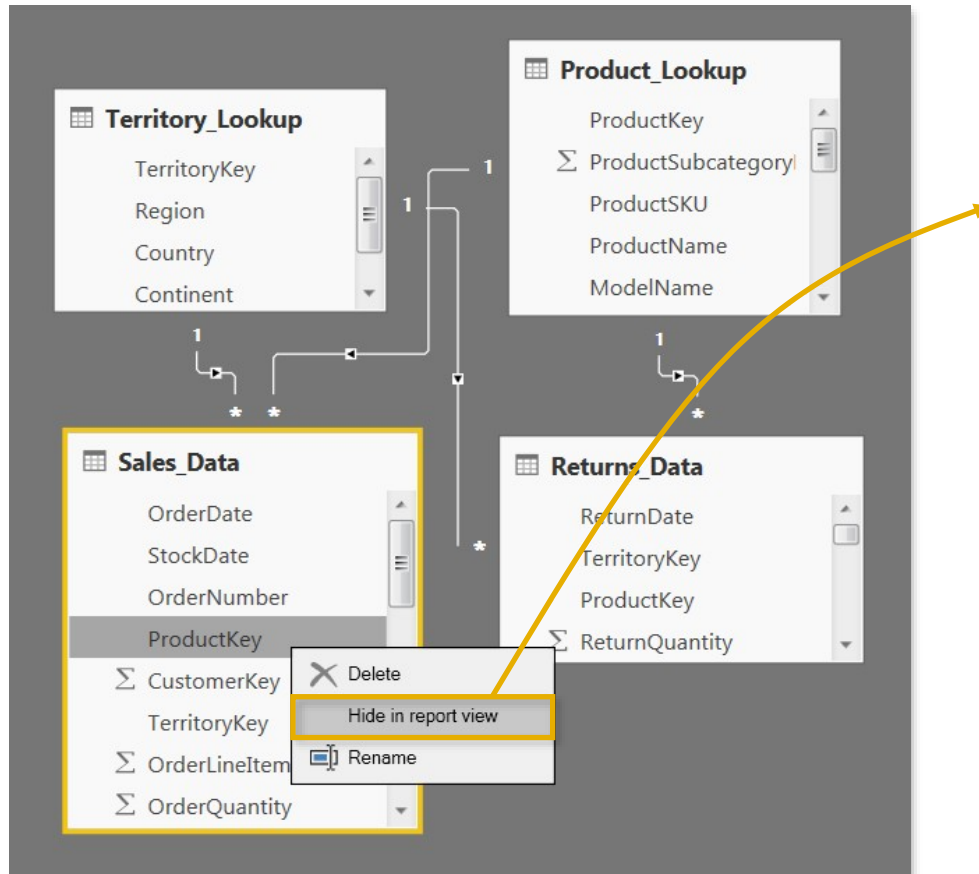
- Territory_Lookup** (Table): TerritoryKey, Region, Country, Continent
- Sales Data** (Table): OrderDate, StockDate, OrderNumber, ProductKey, CustomerKey, TerritoryKey, OrderLineItem, OrderQuantity
- Returns Data** (Table): ReturnDate, TerritoryKey, ProductKey, ReturnQuantity

Relationships:

- Territory_Lookup (1) to Sales Data (Many) on TerritoryKey
- Territory_Lookup (1) to Returns Data (Many) on TerritoryKey

- from “Single” to “Both” allow you to filter context to flow both ways.
- Those measures are applied to the Sales, Data table will pass to the lookups, and then down to the Returns Data table

Hide Fields From Report View



Calculated field With Dax

Data Analysis Expressions, known as (DAX), is formula language that is used in Power BI. With DAX, you can:

- **Simply Add calculated columns and measure to your models, using intuitive syntax.**
 - **the capabilities of traditional “grid-style” formulas, with more powerful and flexible functions.**
- built specifically to work with relations data models**

Two ways to use DAX

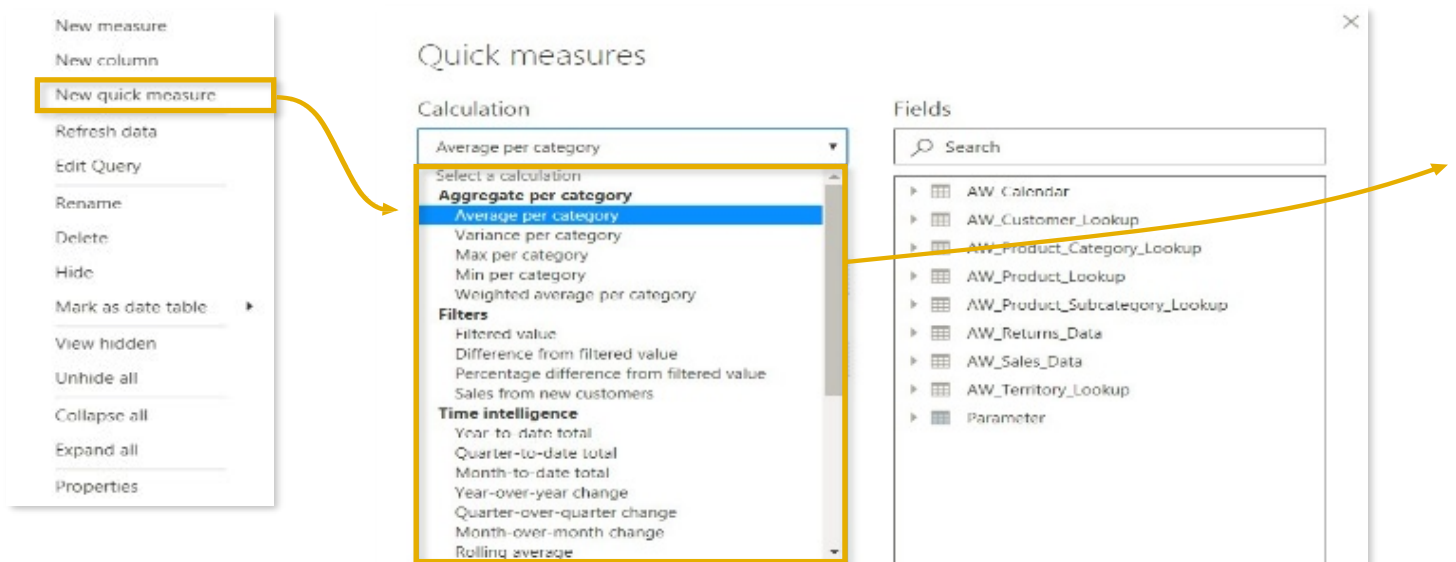
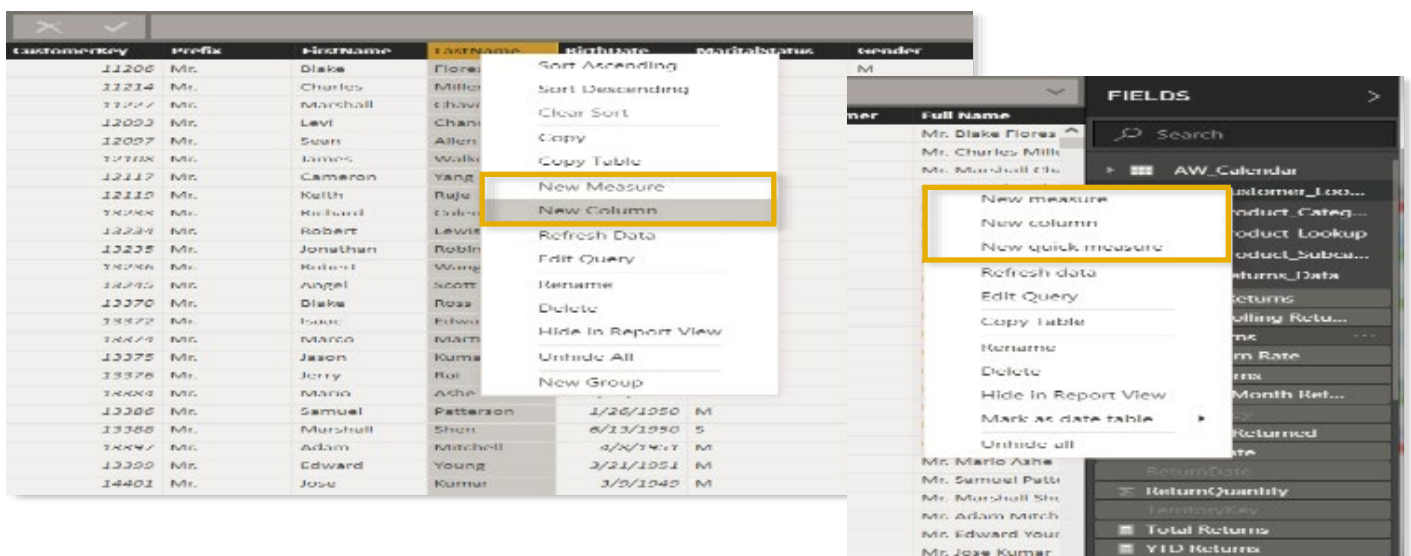
The screenshot displays the Power BI interface with a data table and a list of DAX formulas. The data table has columns: EducationLevel, Occupation, HomeOwner, Full Name, User Name, Domain, IncomeLevel, and Parent. The Parent column contains 'Yes' or 'No' values. The DAX formulas are:

- Parent** = IF(Customer_Lookup[TotalChildren]>0, "Yes", "No")
- Total Orders** = DISTINCTCOUNT(Sales_Data[OrderNumber])
- Total Revenue** = SUMX(Sales_Data, Sales_Data[OrderQuantity] * RELATED(Product_Lookup[ProductPrice]))
- Quantity Ordered** = SUM(Sales_Data[OrderQuantity])

Measures

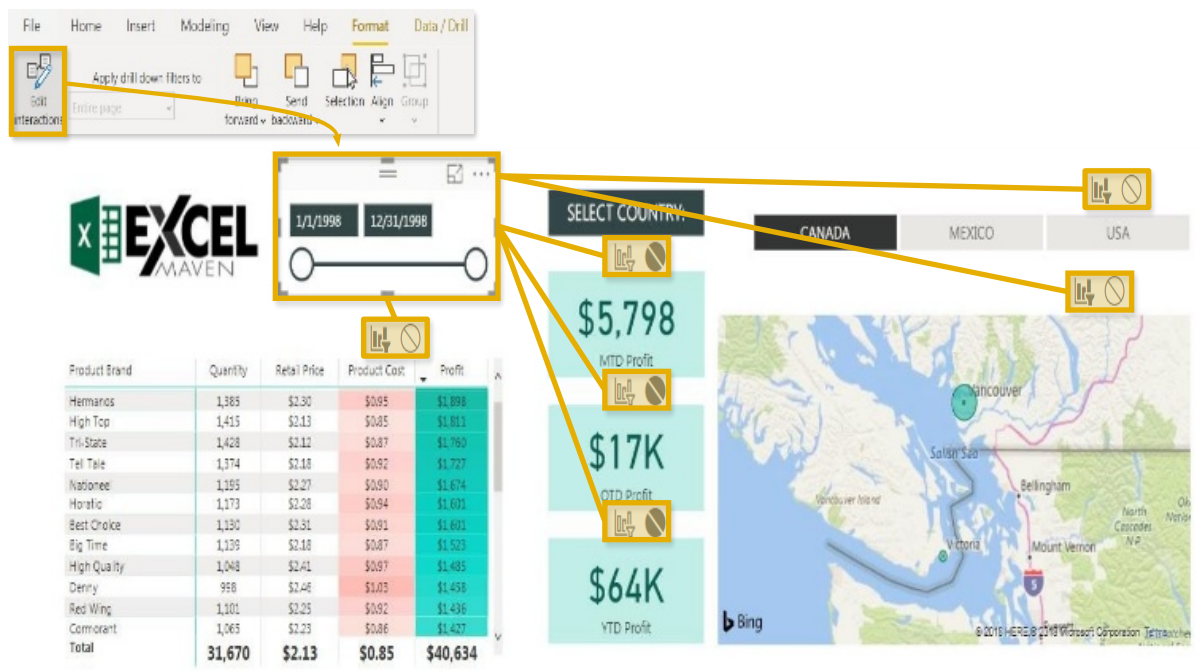
Whenever you are insert Columns or Measures using the **Home** tab (Option 1), they are assigned to table is *currently selected*, or the *first table in the field list* by default

- Measures can reassigned to new “Home” tables (under the “*Structure*” options in the contextual **Measure Tools**), but Option 2 allows you to be more deliberate about placing them
- Assigning measures a to specific table doesn’t have ANY impact on functionality – it’s just a way to keep them organized



A Simple Quick Measures is a pre-made formula template that allows you to drag and drop fields instead of creating DAX from scratch.

These tools **help define** more complex **metrics such as** weighted averages **and** time intelligence **formulas. It encourages** laziness **and doesn't help you understand the basics of Dax.**



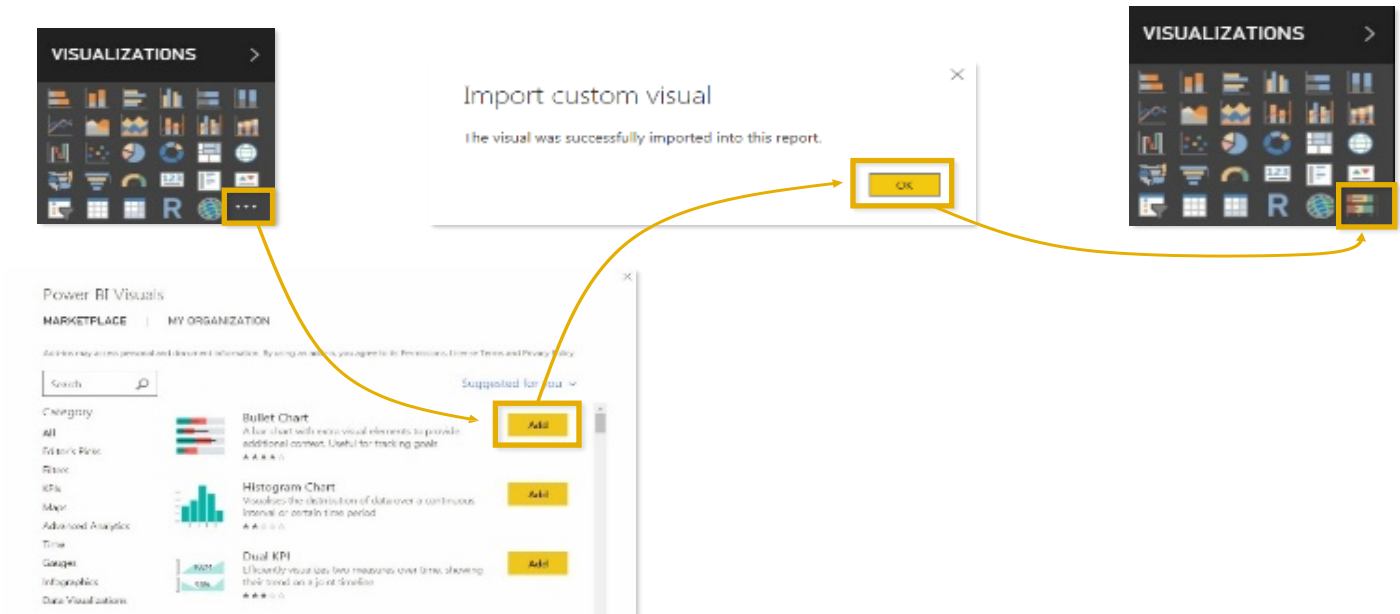
Use there port to determine how filters applied to one visual affect other visuals.

You can manually determine, for example, by selecting a timeline visual and checking

Edit Interactions in the Format tab

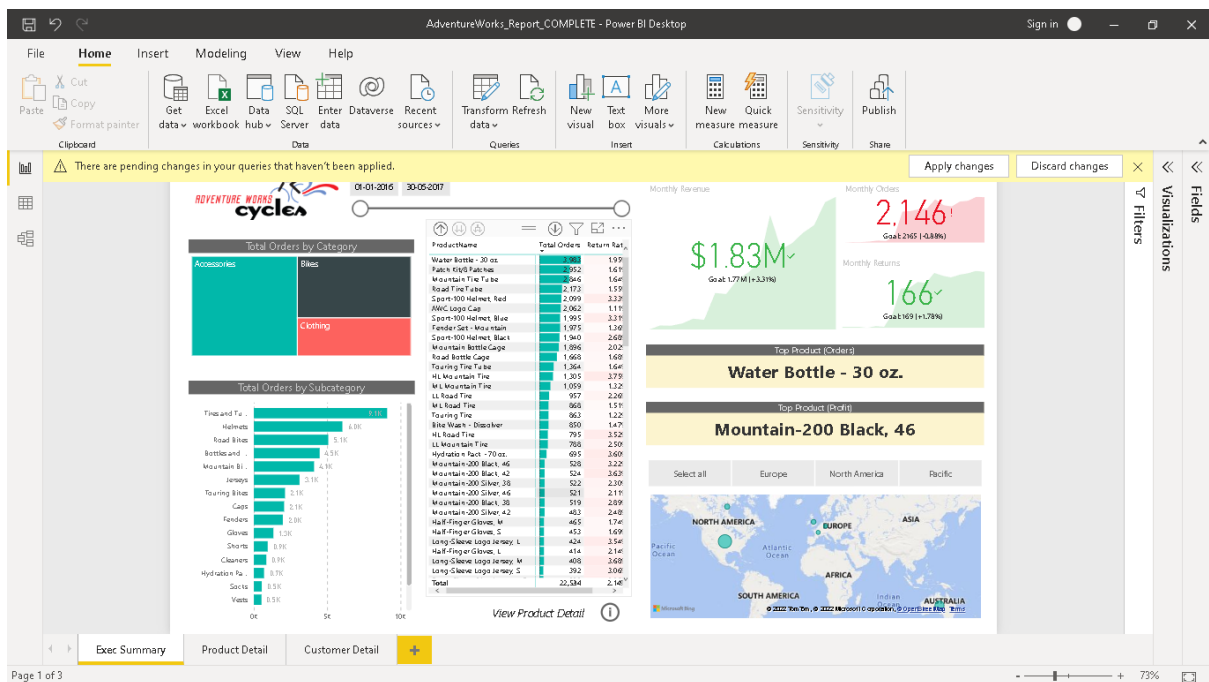
When the date range changes the visual from File or PowerBI Marketplace
Which visuals "react" directly in the report (no installation or reboot required)

In this case I added a **bullet point** from the marketplace

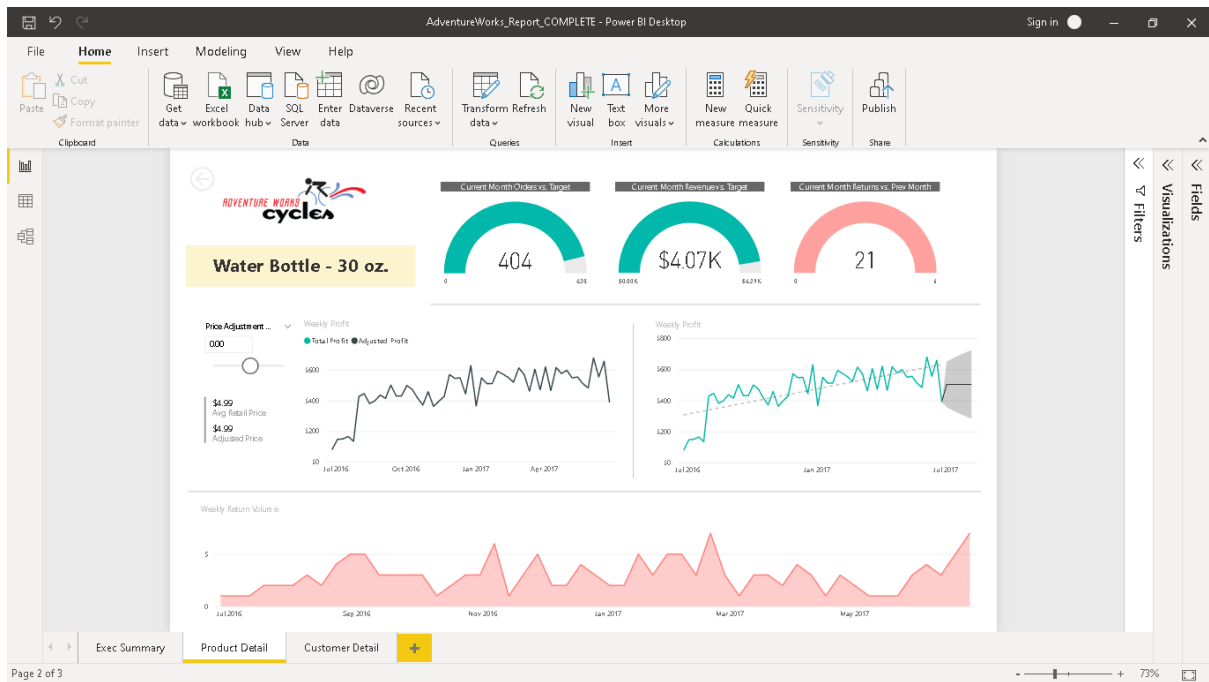


Snapshots

Dashboard



Product Details



Customer Detail

